ARTIFICIAL INTELLIGENCE + MACHINE LEARNING

Concise revision notes, key concepts, and Week 1 practical guide

Prepared from the supplied *Ai_Notes.pdf*, *practice set.pdf*, and *week 1.pdf*. The AI notes are a broad 211-page course handout; the two Week 1 documents focus on ML fundamentals, data preprocessing, basic regression, and a Titanic data-cleaning mini project.

What this pack contains

- A structured summary of the AI handout: problem solving/search, logic, fuzzy logic, knowledge representation, matching, NLP, expert systems, and information retrieval.
- A Week 1 ML cheat sheet: learning types, dataset exploration, missing-value handling, encoding, scaling, splitting, and linear regression.
- A practical checklist for the Titanic Survival Prediction – Data Cleaning Project.

1. Artificial Intelligence: overview

The handout describes AI as the science and engineering of building computer systems that exhibit abilities associated with intelligence, such as reasoning, learning, understanding language, recognizing patterns, planning, and solving problems. AI systems typically represent relevant knowledge and use search, inference, rules, or learned models to reach goals.

Perspectives and goals

- **Intelligence perspective:** systems act in ways people associate with intelligent behaviour. The Turing Test is presented as a test based on whether a human evaluator can distinguish a machine from a human through responses.
- **Business perspective:** AI methods are tools for solving business problems, such as customer segmentation, fraud detection, and classification.
- **Programming perspective:** symbolic representations, search, and problem-solving are central. The notes mention LISP and PROLOG as historically important AI languages.
- **Scientific goal:** understand which ideas explain natural intelligence. **Engineering goal:** use AI techniques to solve real-world problems.

Applications listed in the notes

- Game playing and search; speech recognition and production; natural-language understanding and machine translation; computer vision; expert systems; heuristic classification; consumer marketing/data mining; biometric identification; intrusion detection.

Related disciplines and subfields

The handout traces AI roots to philosophy, logic and mathematics, computation, psychology/cognitive science, biology/neuroscience, and evolution. Subfields include neural networks, evolutionary computation, vision, robotics, expert systems, speech processing, NLP, planning, and machine learning.

Common AI techniques

- **Knowledge representation:** encode facts and relationships in a usable form, for example with logic, rules, frames, semantic networks, or neural-network weights.
- **Learning:** build or adjust knowledge from examples or experience.
- **Rule systems:** explicitly encoded if–then rules or rules implicitly captured by a learned model.
- **Search:** explore possible states, actions, or model configurations to find a solution.

2. Problem solving and search

A search problem can be described by an initial state, possible actions/transitions, a state space, a goal test, and sometimes path costs. A solution is a path from the initial state to a goal. Realistic state spaces can be too large to store explicitly, so algorithms decide which nodes to expand.

How to evaluate a search algorithm

- **Completeness:** will it find a solution if one exists?
- **Optimality:** will it find the best/lowest-cost solution?
- **Time complexity:** how much computation is required?
- **Space complexity:** how much memory is required?

Uninformed search methods

Method	Core idea	Typical strengths / limitations
BFS	Expand shallowest nodes first; commonly uses a FIFO queue.	Complete and optimal when step costs are equal; high memory use.
DFS	Explore one branch deeply before backtracking; commonly uses a stack/recursion.	Low memory relative to BFS; not generally complete/optimal in unbounded spaces.

Method	Core idea	Typical strengths / limitations
Depth-limited search	DFS with a maximum depth limit.	Terminates at the limit; may miss deeper goals.
Iterative deepening	Repeat depth-limited DFS with increasing limits.	Combines DFS-like memory with BFS-like completeness/optimality for equal costs.
Bidirectional search	Search forward from start and backward from goal until frontiers meet.	Can greatly reduce search depth; needs a usable reverse search and known goal.

Informed search and optimization

- **Heuristic:** an estimate or rule of thumb that guides search toward promising states.
- **Greedy best-first search:** prioritizes the node estimated to be closest to the goal; speed depends on heuristic quality and it is not generally optimal.
- **A*:** evaluates a node using **f(n) = g(n) + h(n)**, where g is the cost already paid and h estimates the remaining cost. An admissible heuristic never overestimates the remaining cost; under the relevant conditions, A* is optimal.
- **Hill climbing:** repeatedly moves to an improved neighbouring state; can stop at a local optimum or plateau.
- **Branch and bound:** uses bounds to prune partial solutions that cannot beat the best solution found so far.
- **AO*:** handles AND-OR graphs, where an AND node requires solving all required successors and an OR node can be solved through one suitable successor.
- **Constraint Satisfaction Problems (CSPs):** search for assignments satisfying constraints, e.g., placing queens so no pair shares a row, column, or diagonal.

Repeated states

Search may revisit the same state through different paths or create cycles. The notes discuss preventing immediate reversals, preventing cycles along a path, and remembering previously generated states. The last approach can reduce repeated work but uses additional memory.

3. Formal logic and inference

The AI handout covers propositional logic and first-order predicate logic as formal ways to represent facts and derive conclusions.

Propositional logic

- A proposition is a statement that is either true or false. Common connectives: NOT (¬), AND (∧), OR (∨), implication (→), and biconditional (↔).
- An implication A → B is false only when A is true and B is false.
- **Modus ponens:** from A → B and A, infer B.
- A well-formed formula (WFF) follows the syntax rules of the logic.

Predicate / first-order logic

- Predicates describe properties or relationships, e.g., Student(x). Variables refer to objects in a domain.
- Universal quantifier ∀ means 'for every'; existential quantifier ∃ means 'there exists'.
- Carefully track variable scope and quantifier order when translating natural-language statements.

Clause form and resolution

The notes include conversion of formulas to clausal form and the resolution principle. In broad terms, formulas are transformed into a standard clause representation so complementary literals can be combined to derive consequences or demonstrate contradiction. Typical preparation steps include removing ↔ and →, moving negations inward, standardizing variables, Skolemizing existential variables where appropriate, dropping universal quantifiers, and distributing OR over AND.

Inconsistency and uncertainty

Classical logic can produce problematic conclusions when inconsistent facts are present; the course outline explicitly includes handling inconsistencies and uncertainty. Fuzzy logic represents degrees of membership rather than only crisp true/false membership.

4. Fuzzy logic

A fuzzy set assigns each element x a membership value μA(x) in [0,1]. A value of 0 means no membership, 1 means full membership, and intermediate values represent partial membership. This differs from a crisp set, where membership is binary.

Operations and inference

- Common fuzzy-set operations include union, intersection, and complement/difference. The handout illustrates max/min-based operations and relation composition.
- A membership function maps an input value to a degree of membership in a linguistic category such as 'low', 'medium', or 'high'.
- A fuzzy inference system typically fuzzifies inputs, applies rules, combines outputs, and may defuzzify the result.
- **Defuzzification** converts a fuzzy output into a single crisp value for decision-making.

For exam answers, be able to define membership functions, distinguish crisp and fuzzy sets, explain fuzzy AND/OR, and outline the role of defuzzification.

5. Knowledge representation and organization

Knowledge representation makes information computer-tractable so an AI system can reason or act on it. The notes cover structures such as semantic networks/graphs, frames, and related knowledge organization and manipulation techniques.

Core ideas

- **Semantic networks / graphs:** represent concepts or objects as nodes and relations as links.
- **Frames:** structured descriptions of objects or situations using slots/attributes and their values; they can capture defaults and inheritance.
- **Rules:** represent knowledge as condition–conclusion or IF–THEN statements.
- **Ontology:** describes the kinds of entities that exist in a domain and their properties/relations.
- **Similarity and matching:** compare structures or attributes; quantitative descriptions can use distance metrics to estimate closeness.

Knowledge organization and retrieval

The notes discuss organizing knowledge for efficient access, including indexing and structured records. The practical goal is to make retrieval and reasoning efficient while preserving meaningful relationships among facts.

6. Matching techniques

Matching compares an input pattern or description against stored patterns, templates, or knowledge structures. The handout connects matching with similarity measures and pattern recognition.

- Exact matching checks whether representations agree directly.
- Approximate matching allows partial similarity where descriptions are noisy, incomplete, or expressed differently.
- For numerical features, distance measures can quantify closeness; feature scaling matters when attributes use different units.
- Pattern recognition uses observed features to classify or identify an object, event, text pattern, or other input.

7. Natural Language Processing (NLP)

NLP aims to process human language, including understanding, parsing, and translation. The notes stress that recognizing words or parsing grammar alone is not enough: a system also needs knowledge of the domain and meaning.

Parsing and language understanding

- A grammar describes valid structures; parsers determine how a sentence fits those structures.
- Transition networks are described as finite-state structures used to represent parts of a grammar.
- Language is ambiguous: words and sentences can have multiple meanings, so context and domain knowledge matter.
- Machine translation must preserve meaning, not merely replace each word with a word in another language.

Information retrieval connection

The later material discusses search engines and information retrieval. Retrieval systems match user queries to documents; useful approaches can find relevant documents even when they do not contain the exact words used in a query.

8. Expert systems

An expert system uses domain knowledge and inference procedures to make recommendations or solve a restricted class of problems. A knowledge engineer elicits knowledge from domain experts and encodes it into a knowledge base.

Main components

- **Knowledge base:** facts and domain rules.
- **Inference engine:** applies rules to known facts to derive conclusions.
- **User interface / explanation facility:** accepts input and may explain how a conclusion was reached.
- **Knowledge acquisition:** captures and updates expert knowledge.

Strengths and limitations

Expert systems can perform well in a narrow, clearly represented domain, but their usefulness depends on the completeness and suitability of the knowledge representation. They may lack common-sense knowledge and can fail outside their intended scope. The handout names MYCIN as an early medical expert system for bacterial blood infections.

Forward and backward reasoning

- **Forward chaining:** start from known facts and fire rules to derive new facts/conclusions.
- **Backward chaining:** start with a goal and work backward to determine which facts or subgoals would prove it.

9. Machine Learning — Week 1

The supplied Week 1 materials focus on ML fundamentals and data preprocessing: importing and exploring datasets, cleaning missing values, feature engineering, encoding, scaling, train/test splitting, and basic linear regression.

Three learning types

- **Supervised learning:** learn from labelled examples (features X and known target y). Examples include regression and classification.
- **Unsupervised learning:** discover structure in unlabelled data, such as clusters or lower-dimensional representations.
- **Reinforcement learning:** an agent learns actions through rewards/penalties while interacting with an environment.

Typical data workflow

1. Load data → 2. inspect rows, types and summary statistics → 3. identify missing/invalid values → 4. impute or remove missing data → 5. encode categorical columns → 6. scale numerical features if needed → 7. split into train/test sets → 8. train a model → 9. evaluate on held-out data → 10. save cleaned data/model outputs.

Pandas exploration

- Use `head(10)` to inspect the first 10 rows.
- Use `info()` to inspect column names, data types, and non-null counts.
- Use `describe()` to summarize numerical columns.

Missing-value handling

- Mean imputation is often used for numeric data without strong skew/outlier problems.
- Median imputation is more robust to skew and outliers.
- Fit imputers using training data only in predictive workflows to avoid information leakage; apply the fitted transformation to test data.

Categorical encoding

- **LabelEncoder:** maps category labels to integer codes; often appropriate for target labels, and should be used carefully for nominal input features because the integers can imply a false order.
- **OneHotEncoder:** creates indicator columns for categories; useful for nominal features such as embarkation port.

Scaling and data leakage

Scaling places numerical features on comparable scales. Fit scaling transformations on the training set and reuse them on test data. Do not use test-set information to make training-time preprocessing decisions.

Linear regression

Linear regression models a continuous target as a linear function of one or more features. For a single feature, the model can be written as ■ = b■ + b■x. Train the model on training data, predict on test data, and evaluate with metrics such as R^2, mean absolute error (MAE), and root mean squared error (RMSE). R^2 is a regression score, not classification accuracy.

House-price exercise

The practice set asks you to load a CSV, split it into train/test subsets, train a Linear Regression model, assess its performance, and predict price from an area input. The code pack includes a clearly labelled sample CSV for testing; replace it with a real dataset for meaningful predictions.

10. Mini Project 1 — Titanic data cleaning

The Week 1 project is titled **“Titanic Survival Prediction – Data Cleaning Project.”** The supplied task list asks you to clean missing data, encode `Sex` and `Embarked`, visualize the age distribution with Matplotlib/Seaborn, and export a cleaned dataset as a new CSV.

Suggested deliverables

- Original dataset file (download the Titanic training CSV from Kaggle; it is not included in the supplied PDFs).
- Script that checks data shape, types, missing counts, and summary statistics.
- Cleaning choices documented: median for numeric Age; most-frequent value for Embarked; a stated strategy for Cabin; drop rows missing the target if training a survival model later.
- Encode Sex and Embarked. The provided project script uses one-hot encoding for Embarked and binary mapping for Sex, retaining readable/traceable columns.
- Save the cleaned data to `titanic_cleaned.csv` and an age-distribution plot to `age_distribution.png`.

Important scope note

The supplied project brief is a **data-cleaning project**; it does not explicitly require training a survival prediction model. The code therefore focuses on cleaning, encoding, visualizing, and exporting. A predictive model can be added as an extension.

11. Quick revision checklist

- Explain AI, its goals, applications, roots, and major subfields.
- Define initial state, state space, actions, goal test, path cost, completeness, optimality, time and space complexity.
- Compare BFS, DFS, depth-limited search, iterative deepening, and bidirectional search.
- Explain heuristics, A* f(n)=g(n)+h(n), admissibility, hill climbing, branch-and-bound, and AO*.

- Translate simple statements into propositional and predicate logic; know quantifiers, implication, Modus Ponens, clause form and resolution.
- Define fuzzy membership, common set operations, fuzzy inference, and defuzzification.
- Compare knowledge graphs/semantic networks, frames, rules, ontologies, and expert systems.
- Explain NLP parsing, ambiguity, machine translation, pattern recognition and retrieval.
- In ML, distinguish supervised, unsupervised, and reinforcement learning.
- Practice pandas exploration, imputation, categorical encoding, train/test split, regression metrics, and CSV export.

Source note: This is a study-oriented summary of the uploaded handout and Week 1 task sheets. Where the source gives only a topic heading rather than a detailed procedure, the summary keeps the explanation introductory and flags implementation choices in the code README.